

# Tutorial\_4\_Browser\_Automation

## Tutorial 4: Browser Automation for Testing

**Duration:** 30 minutes **Level:** Intermediate

### Overview

Automate web testing using HelixCode's browser control: - Launch Chrome/Chromium - Navigate and interact - Take screenshots - Fill forms - Run test scenarios

### Step 1: Setup Browser Tools

```
helixcode browser init

# Downloads Chromium if needed
# Verifies installation
```

### Step 2: Create Test Plan

```
helixcode plan "Create automated E2E tests for login flow:
- Navigate to login page
- Fill username and password
- Submit form
- Verify redirect to dashboard
- Take screenshot
- Check for welcome message"
```

### Step 3: Generate Test Script

```
helixcode generate "Create browser automation test for login"
```

**Generated** tests/e2e/login\_test.go:

```
package e2e

import (
    "testing"
    "dev.helix.code/internal/tools/browser"
)

func TestLoginFlow(t *testing.T) {
    // Launch browser
    br, _ := browser.NewBrowserController(&browser.Config{
        Headless: true,
```

```

}))
defer br.Close()

br.Launch(ctx, &browser.LaunchOptions{})

// Navigate to login
br.Navigate(ctx, "http://localhost:3000/login")

// Fill form
br.Type(ctx, "#username", "testuser")
br.Type(ctx, "#password", "password123")

// Submit
br.Click(ctx, "button[type=submit]")

// Wait for navigation
br.WaitForNavigation(ctx)

// Verify URL
url, _ := br.GetURL(ctx)
if url != "http://localhost:3000/dashboard" {
    t.Errorf("Expected dashboard, got %s", url)
}

// Screenshot
br.Screenshot(ctx, &browser.ScreenshotOptions{
    Path: "screenshots/login-success.png",
})

// Verify welcome message
text, _ := br.GetText(ctx, ".welcome-message")
if text != "Welcome, testuser!" {
    t.Errorf("Expected welcome message, got %s", text)
}
}

```

## Step 4: Web Scraping Example

helixcode generate "Scrape product prices from e-commerce site"

```

// Extract product data
products, _ := br.Evaluate(ctx, `
    Array.from(document.querySelectorAll('.product')).map(p => ({
        name: p.querySelector('.name').textContent,
        price: p.querySelector('.price').textContent,
        image: p.querySelector('img').src
    }))
`)

```

## Step 5: Visual Regression Testing

```
helixcode generate "Create visual regression test suite"

// Take screenshots of all pages
pages := []string{"/", "/about", "/contact", "/products"}

for _, page := range pages {
    br.Navigate(ctx, "http://localhost:3000"+page)
    br.Screenshot(ctx, &browser.ScreenshotOptions{
        Path: fmt.Sprintf("baseline/%s.png", page),
        FullPage: true,
    })
}

// Compare with baseline
// Diff algorithm detects visual changes
```

## Step 6: Run Tests with HelixCode

```
helixcode test --browser --headless

# Runs all browser tests
# Generates report with screenshots
```

## Results

- **Automated Testing:** Complete E2E test suite
- **Visual Regression:** Catch UI bugs automatically
- **CI Integration:** Headless mode for pipelines

---

Continue to [Tutorial 5: Voice-to-Code Workflow](#)

## Sources verified

Sources verified 2026-05-29: <https://pkg.go.dev/github.com/chromedp/chromedp> (chromedp actively maintained, v0.15.x line, Mar 2026; MIT; ~2.1k importers) ; <https://go.dev/dl/> (go1.26.3 latest stable Go) ; project go.mod (inner go 1.26) + CLAUDE.md §3.1 (chromedp = headless-browser dependency). The browser-automation snippets use the project's `dev.helixcode/internal/tools/browser` wrapper over chromedp; no chromedp version pin appears in this tutorial, so nothing was stale to fix. **NEGATIVE FINDING** — `localhost:3000/localhost` endpoints are illustrative app targets, not external services to verify.